



Politechnika Warszawska

Wydział Matematyki
i Nauk Informatycznych

Algorytmy zaawansowane

Komiwojażer

Porównanie dwóch algorytmów aproksymacyjnych
znajdujących rozwiązanie problemu Komiwojażera

Raport końcowy

Zespół:

inż. Marcin Cieszyński

inż. Radosław Głasek

inż. Adam Grącikowski

Prowadzący:

dr Michał Tuczyński

Oświadczamy, że niniejsza praca, stanowiąca podstawę do uznania osiągnięcia efektów uczenia się z przedmiotu *Algorytmy zaawansowane*, została wykonana samodzielnie.

Warszawa, 24 maja 2026

Spis treści

1	Wstęp	2
1.1	Cel projektu	2
1.2	Przedstawienie problemu	2
2	Zmiany w stosunku do dokumentacji wstępnej	3
3	Instrukcja użytkownika	3
3.1	Składnia poleceń	3
3.1.1	Dostępne argumenty i flagi	3
3.2	Przykłady użycia	3
3.3	Format danych wejściowych	4
3.4	Format danych wyjściowych	4
3.5	Przykłady wejścia i wyjścia	4
4	Generowanie instancji testowych	6
4.1	Składnia poleceń	6
4.1.1	Dostępne argumenty i flagi	6
4.2	Przykłady użycia	6
5	Opis eksperymentów	7
5.1	Losowe instancje o małym rozmiarze	7
5.2	Losowe instancje o dużym rozmiarze	7
5.3	Instancje referencyjne z biblioteki TSPLIB	7
5.4	Metodologia pomiarów	7
6	Wyniki i wnioski	8
6.1	Złożoność obliczeniowa i czas wykonania	8
6.1.1	Analiza małych instancji losowych	8
6.1.2	Analiza dużych instancji losowych	9
6.2	Jakość aproksymacji	9
6.3	Analiza statystyczna błędu względnego	10
6.3.1	Weryfikacja hipotez statystycznych	11
6.4	Zależność błędu aproksymacji od rozmiaru problemu	12
6.5	Czas obliczeń a precyzja	12
7	Podsumowanie	14
8	Podział pracy	15

1 Wstęp

Problem komiwojażera (ang. *Traveling Salesperson Problem*, TSP) to jeden z klasycznych problemów optymalizacji kombinatorycznej, należący do klasy problemów NP-trudnych.

1.1 Cel projektu

Przedmiotem projektu jest implementacja oraz szczegółowe porównanie dwóch algorytmów aproksymacyjnych rozwiązujących metryczną wersję problemu komiwojażera. Porównanie ma na celu ewaluację obu podejść pod kątem jakości zwracanych rozwiązań w stosunku do czasu ich wykonania na instancjach testowych o zróżnicowanej wielkości [1].

1.2 Przedstawienie problemu

Zadanie komiwojażera polega na znalezieniu najkrótszej możliwej trasy, która zaczyna się w wybranym punkcie, odwiedza dokładnie jeden raz każdy z pozostałych punktów, a następnie wraca do miejsca startu.

W ujęciu grafowym problem ten modeluje się za pomocą pełnego grafu nieskierowanego ważonego $G = (V, E)$, gdzie V to zbiór wierzchołków reprezentujących lokalizacje, a E to zbiór krawędzi łączących każdą parę wierzchołków. Każdej krawędzi przypisana jest waga za pomocą funkcji kosztu $c : E \rightarrow \mathbb{N}$. Zastosowanie dziedziny liczb naturalnych oznacza, że operujemy na całkowitoliczbowych, nieujemnych wartościach odległości. Naszym celem jest znalezienie cyklu Hamiltona C o minimalnym sumarycznym koszcie krawędzi.

Wersja metryczna problemu, którą rozpatrujemy w tym projekcie, nakłada na funkcję kosztu dodatkowe ograniczenia, upodabniające graf do rzeczywistej przestrzeni fizycznej. Najważniejszym z nich jest **nierówność trójkąta**. Oznacza ona, że bezpośrednia podróż z wierzchołka u do w nigdy nie jest droższa niż podróż przez jakikolwiek wierzchołek pośredni v . Formalnie dla każdych trzech wierzchołków $u, v, w \in V$ zachodzi:

$$c(u, w) \leq c(u, v) + c(v, w)$$

Dodatkowo zakładamy symetrię, czyli $c(u, v) = c(v, u)$.

2 Zmiany w stosunku do dokumentacji wstępnej

Nie wprowadzono zmian w stosunku do dokumentacji wstępnej.

3 Instrukcja użytkownika

Aplikacja została zrealizowana w środowisku .NET 9 jako program konsolowy, który wczytuje instancję problemu z pliku tekstowego. Wynik działania algorytmu jest domyślnie wypisywany na standardowe wyjście (konsolę) w czytelnej dla użytkownika formie, a opcjonalnie może zostać zapisany do pliku tekstowego.

3.1 Składnia poleceń

Podstawowa składnia wywołania programu (zakładając, że skompilowany plik wykonywalny nosi nazwę `tsp-approximation.exe`) prezentuje się następująco:

```
tsp-approximation.exe <FilePath> [opcje]
```

3.1.1 Dostępne argumenty i flagi

Program przyjmuje jeden argument pozycyjny oraz dodatkowe flagi modyfikujące jego zachowanie:

- `<FilePath>` (argument wymagany) – argument pozycyjny podawany bezpośrednio po nazwie programu. Określa ścieżkę do pliku tekstowego zawierającego macierz sąsiedztwa grafu.
- `-o, --output <ścieżka>` (argument opcjonalny) – pozwala na zdefiniowanie ścieżki do pliku wyjściowego, w którym zapisany zostanie wynik działania wybranego algorytmu. Jeśli parametr ten zostanie pominięty, wynik zostanie wypisany wyłącznie na standardowe wyjście (konsolę).
- `-d, --double-tree` (flaga opcjonalna) – jej podanie wymusza użycie algorytmu 2-aproksymacyjnego. Jeśli flaga nie zostanie ustawiona, program domyślnie wykorzysta algorytm Christofidesa.
- `--help` – wbudowana flaga, która wyświetla na ekranie krótką instrukcję obsługi i opisy dostępnych argumentów w języku angielskim.

3.2 Przykłady użycia

Uruchomienie domyślne (algorytm Christofidesa):

```
tsp-approximation.exe graph.txt
```

Program wczyta graf z pliku `graph.txt`, wykona obliczenia za pomocą algorytmu Christofidesa, a wynik zaprezentuje bezpośrednio w konsoli.

Uruchomienie algorytmu podwójnego drzewa z zapisem do pliku:

```
tsp-approximation.exe graph.txt -d -o result.txt
```

Program wykorzysta algorytm 2-aproksymacyjny (`-d`) i zapisze sformatowany cykl Hamiltona oraz jego koszt do pliku `result.txt` (`-o`).

Alternatywny zapis z pełnymi nazwami flag:

```
tsp-approximation.exe graph.txt --double-tree --output result.txt
```

Wywołanie to jest w pełni równoważne z przykładem drugim, ale wykorzystuje długie aliasy poleceń.

3.3 Format danych wejściowych

Plik wejściowy jest standardowym plikiem tekstowym. Jego struktura opiera się na następujących zasadach:

- W pierwszej linii znajduje się pojedyncza liczba całkowita dodatnia N , określająca całkowitą liczbę wierzchołków w grafie.
- W kolejnych N liniach znajduje się macierz sąsiedztwa modelująca graf pełny. Każdy wiersz zawiera dokładnie N wartości oddzielonych białymi znakami (pojedynczą spacją lub tabulatorem).

Wartości w macierzy reprezentują wagi krawędzi (odległości) i są liczbami naturalnymi. Macierz jest symetryczna, posiada zera na głównej przekątnej, a zapisane w niej odległości spełniają nierówność trójkąta.

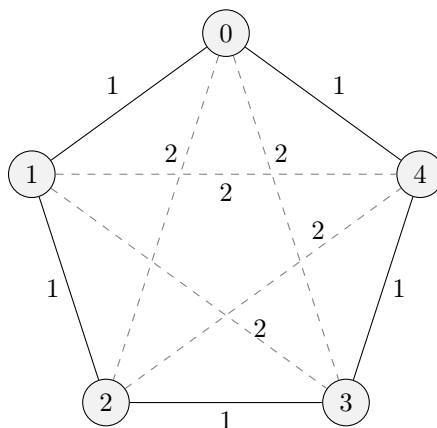
3.4 Format danych wyjściowych

Plik wyjściowy jest standardowym plikiem tekstowym. Jego struktura opiera się na następujących zasadach:

- Pierwsza linia rozpoczyna się od frazy „Total Distance: ”, po której następuje całkowity koszt znalezionej rozwiązania (suma wag krawędzi tworzących cykl Hamiltona).
- Druga linia rozpoczyna się od frazy „Route: ”, po której następuje reprezentacja wyznaczonego cyklu Hamiltona. Identyfikatory wierzchołków wypisane są w kolejności odwiedzania i rozdzielone znakiem pauzy (-). Pierwszy i ostatni wierzchołek w tej linii są identyczne, domykając cykl.

3.5 Przykłady wejścia i wyjścia

Przeanalizujemy zawartość pliku wejściowego oraz wyjściowego na przykładzie grafu G przedstawionego na Rysunku 1.



Rysunek 1: Przykładowy graf wejściowy G .

Graf składa się z pięciu wierzchołków ($N = 5$). Krawędzie ułożone wzdłuż obwodu pięciokąta mają wagę 1, natomiast wszystkie „przekątne” (łącznie wierzchołki niebędące bezpośrednimi sąsiadami na obwodzie) mają wagę 2. Warto zauważyć, że wszystkie wagi krawędzi to liczby naturalne i spełniona jest tu nierówność trójkąta.

Zawartość pliku wejściowego dla grafu G przedstawia Rysunek 2.

```

5
0 1 2 2 1
1 0 1 2 2
2 1 0 1 2
2 2 1 0 1
1 2 2 1 0

```

Rysunek 2: Zawartość pliku wejściowego dla grafu G .

Wynik działania programu dla powyższej instancji został zilustrowany na Rysunku 3. Pierwsza linia pliku wyjściowego precyzuje całkowity koszt znalezionej trasy, wynoszący w tym przypadku 5. Jest to suma wag pięciu krawędzi wchodzących w skład wyznaczonego cyklu. Druga linia określa dokładną kolejność odwiedzania wierzchołków.

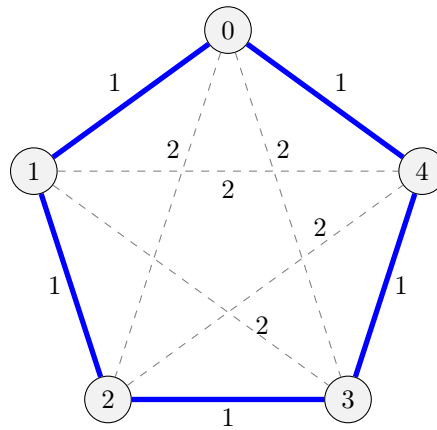
```

Total Distance: 5
Route: 0-1-2-3-4-0

```

Rysunek 3: Zawartość pliku wyjściowego dla grafu G .

W prezentowanym przykładzie wyznaczona ścieżka (0-1-2-3-4-0) przebiega wyłącznie po krawędziach o wadze 1 (Rysunek 4).



Rysunek 4: Rozwiązanie zapisane w pliku wyjściowym.

4 Generowanie instancji testowych

Aplikacja została zrealizowana w środowisku .NET 9 jako program konsolowy.

4.1 Składnia poleceń

Podstawowa składnia wywołania programu prezentuje się następująco:

```
tsp-generator.exe --start <wartość> [opcje]
```

4.1.1 Dostępne argumenty i flagi

Program wymaga podania rozmiaru startowego oraz przyjmuje szereg argumentów opcjonalnych modyfikujących proces generowania:

- **--start <wartość>** (argument wymagany) – określa początkową liczbę wierzchołków dla pierwszej instancji w generowanej serii.
- **--step <wartość>** (argument opcjonalny, domyślnie: 1) – rozmiar kroku, o jaki zwiększana jest liczba wierzchołków w każdej kolejnej generowanej instancji.
- **-c, --count <wartość>** (argument opcjonalny, domyślnie: 1) – łączna liczba instancji (plików) do wygenerowania w ramach jednego uruchomienia.
- **-s, --size <wartość>** (argument opcjonalny, domyślnie: 1000) – maksymalna wartość współrzędnych (na osiach X i Y) dla losowanych wierzchołków.
- **-d, --dir <ścieżka>** (argument opcjonalny, domyślnie: **data**) – ścieżka do katalogu docelowego, w którym zostaną zapisane pliki z rozszerzeniem **.txt**.
- **--help** – wyświetla na standardowym wyjściu instrukcję obsługi programu.

4.2 Przykłady użycia

Uruchomienie podstawowe (pojedyncza instancja)

```
tsp-generator.exe --start 50
```

Program wygeneruje pojedynczy graf składający się z 50 wierzchołków, wylosuje współrzędne z przedziału od 0 do 1000 i zapisze wynik w domyślnym katalogu **data**.

Generowanie serii instancji testowych

```
tsp-generator.exe --start 100 --step 10 -c 10 -d test_set
```

Polecenie wygeneruje serię 10 plików (**-c 10**), zaczynając od rozmiaru 100 wierzchołków (**--start 100**) i zwiększając go o 10 w każdym kolejnym kroku (**--step 10**). Pliki zostaną zapisane w katalogu **test_set** (**-d**).

Generowanie na mniejszej płaszczyźnie współrzędnych

```
tsp-generator.exe --start 20 --size 500
```

Program wygeneruje graf 20-wierzchołkowy, w którym maksymalna wartość współrzędnych punktów na osiach X i Y nie przekroczy 500 (**-size**).

5 Opis eksperymentów

W celu przeprowadzenia kompleksowej analizy oraz walidacji zaimplementowanych algorytmów aproksymacyjnych, przygotowano zestaw instancji testowych. Zbiór ten został podzielony na trzy kategorie (znajdujące się w katalogu `examples/`):

- Losowe instancje o małym rozmiarze (`examples/random/small`).
- Losowe instancje o dużym rozmiarze (`examples/random/large`).
- Instancje referencyjne z biblioteki TSPLIB [4, 2] (`examples/tsplib`).

5.1 Losowe instancje o małym rozmiarze

Pierwsza grupa testowa obejmuje 41 wygenerowanych losowo instancji problemu, dla których liczba wierzchołków n zawiera się w przedziale od $n = 10$ do $n = 50$. Pliki z danymi przyjmują konwencję nazewnictwa `gen_n{liczba_wierzchołkow}.txt` (np. `gen_n0010.txt`). Ze względu na minimalny krok przyrostu rozmiaru instancji, wynoszący 1 wierzchołek, zbiór ten jest wykorzystywany do szczegółowej analizy zachowania algorytmów na grafach o niewielkich rozmiarach.

5.2 Losowe instancje o dużym rozmiarze

Druga grupa testowa składa się z 41 wygenerowanych losowo instancji, obejmujących grafy o rozmiarach od $n = 100$ do $n = 500$ wierzchołków. W tym przypadku krok przyrostu wynosi 10 wierzchołków. Konwencja nazewnictwa plików jest tożsama z instancjami małymi (np. `gen_n0100.txt`). Zbiór ten ma na celu weryfikację skalowalności oraz analizę złożoności czasowej algorytmów.

5.3 Instancje referencyjne z biblioteki TSPLIB

Trzecią grupę testową stanowi 40 referencyjnych instancji problemu komiwojażera pochodzących z biblioteki TSPLIB. Pliki w tej podgrupie wykorzystują konwencję:

`n{liczba_wierzchołkow}o{dystans_optymalny}.txt`

Taki format (np. `n51o426.txt`) pozwala na automatyczne odczytanie długości optymalnej trasy bezpośrednio z nazwy pliku.

5.4 Metodologia pomiarów

W ramach przyjętej metodologii badawczej, każda z wyżej opisanych instancji testowych została przetworzona przy użyciu obu analizowanych algorytmów: algorytmu 2-aproksymacyjnego (bazującego na podwójnym drzewie rozpinającym) oraz algorytmu Christofidesa.

Architektura przeprowadzanych eksperymentów numerycznych została zaprojektowana w sposób umożliwiający ewaluację dwóch wskaźników wydajności:

- **Złożoność obliczeniowa:** Czas wykonania mierzony jest przekrojowo na seriach losowych instancji (małych i dużych). Podejście to pozwala na zaobserwowanie dynamiki wzrostu czasu wykonania algorytmów w funkcji rosnącej liczby wierzchołków badanych grafów.
- **Jakość aproksymacji:** Weryfikowana z wykorzystaniem instancji z rodziny TSPLIB. Dzięki znajomości wartości optimum globalnego, możliwe jest precyzyjne określenie, na ile wygenerowane przez algorytmy rozwiązanie przybliżone odbiega od optymalnego.

6 Wyniki i wnioski

W poniższym rozdziale zaprezentowano zestawienie oraz interpretację wyników uzyskanych w trakcie testowania zaimplementowanych algorytmów. Analiza została podzielona na dwa główne aspekty: badanie złożoności obliczeniowej oraz ocenę jakości otrzymywanych rozwiązań (współczynnika aproksymacji).

6.1 Złożoność obliczeniowa i czas wykonania

Pomiary czasu wykonania (wyrażone w milisekundach) pozwalają na weryfikację teoretycznych złożoności obliczeniowych oraz ocenę narzutu czasowego wynikającego ze specyfiki poszczególnych operacji składowych badanych algorytmów.

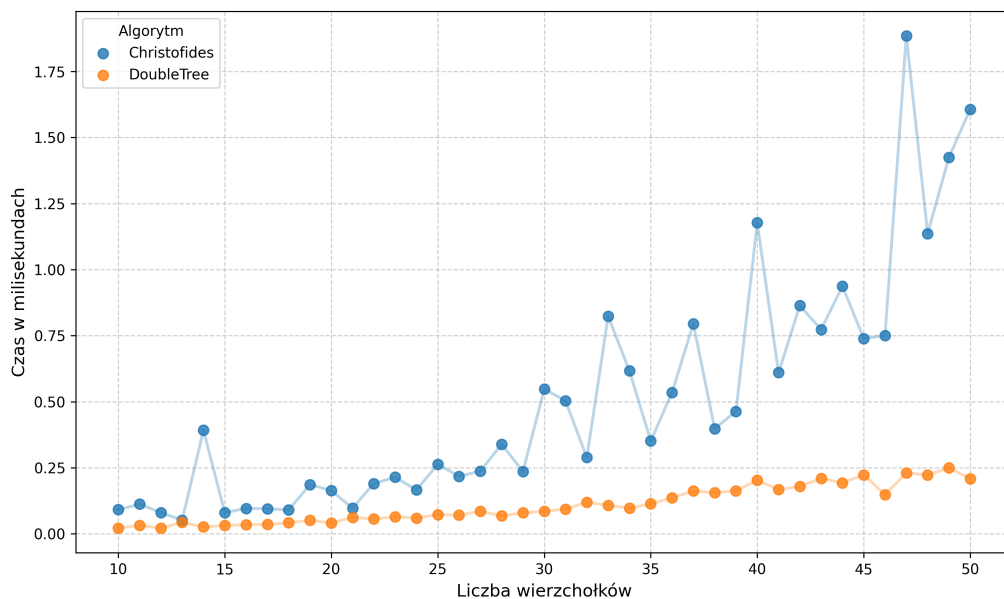
6.1.1 Analiza małych instancji losowych

Na podstawie przeprowadzonych eksperymentów dla zbioru instancji małych, można zaobserwować wyraźne różnice w charakterystyce działania obu badanych algorytmów. Wyniki pomiarów dla tej grupy zostały zagregowane na wykresie porównawczym (Rysunek 5).

Algorytm 2-aproksymacyjny charakteryzuje się bardzo krótkim i relatywnie stabilnym czasem działania. W badanym przedziale od $n = 10$ do $n = 50$ wierzchołków, czas wykonania rośnie w sposób łagodny i przewidywalny, osiągając w wartościach szczytowych około 0,25 milisekundy dla $n = 49$. Zmienność czasu pomiędzy kolejnymi iteracjami (pojedynczymi przyrostami liczby wierzchołków) jest minimalna. Świadczy to o niskim koszcie obliczeniowym podstawowych kroków tego algorytmu – wyznaczania minimalnego drzewa rozpinającego (MST).

Z kolei algorytm Christofidesa okazuje się zauważalnie bardziej wymagający pod względem czasu procesora, co uwiadamia się nawet w przypadku tak niewielkich grafów. O ile dla najmniejszych instancji ($n < 25$) czasy wykonania obu algorytmów pozostają zbliżone i rzadko przekraczają 0,3 ms, o tyle w miarę wzrostu rozmiaru problemu, krzywa algorytmu Christofidesa ulega silnemu rozwarstwieniu względem algorytmu 2-aproksymacyjnego. Dodatkowo, heurystyka ta charakteryzuje się wysoką wariancją czasu wykonania — na wykresie widoczne są gwałtowne skoki, na przykład dla $n = 40$ (niemal 1,2 ms) czy $n = 47$ (blisko 1,9 ms).

Zaobserwowana dysproporcja wynika bezpośrednio z różnic w strukturze obu metod. Algorytm Christofidesa, oprócz konstrukcji drzewa rozpinającego, narzuca konieczność wyznaczenia skojarzenia doskonałego o minimalnej wadze dla podgrafu indukowanego przez wierzchołki o nieparzystych stopniach. Krok ten, w naszej implementacji, charakteryzuje się złożonością rzędu $O(n^4)$ [1]. Zauważalne na wykresach fluktuacje czasu działania algorytmu Christofidesa potwierdzają silną zależność tego etapu od topologii konkretnej instancji losowej (m.in. od liczby wierzchołków nieparzystych powstałych w kroku generowania MST).



Rysunek 5: Porównanie czasu wykonania algorytmów Christofidesa i 2-aproksymacyjnego w funkcji liczby wierzchołków dla małych instancji losowych ($n \in [10, 50]$).

6.1.2 Analiza dużych instancji losowych

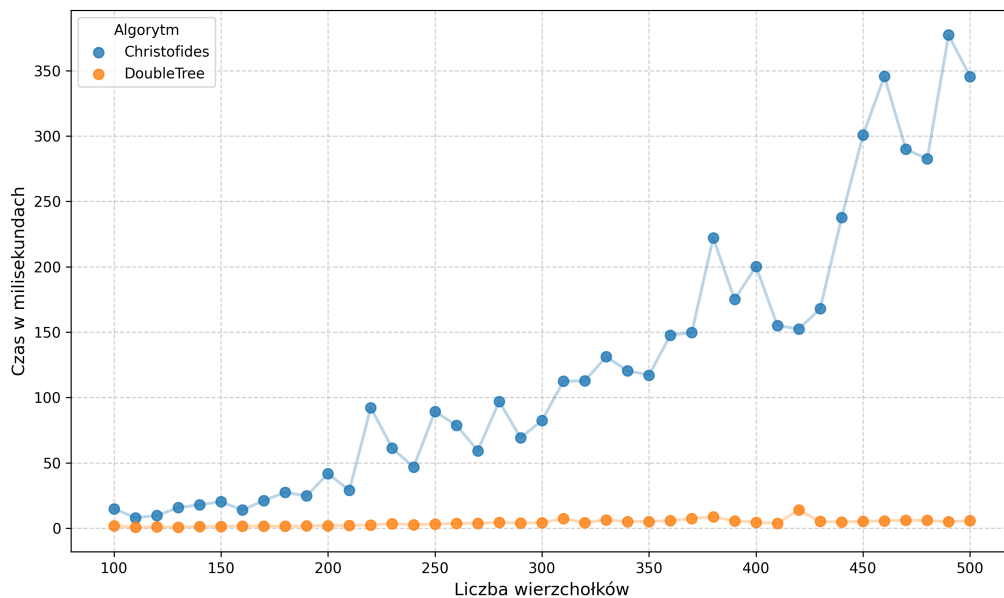
Przejście do analizy grafów o znacznie większej liczbie wierzchołków pozwala zaobserwować asymptotyczne różnice w złożoności obliczeniowej obu zaimplementowanych metod. Zestawienie wyników dla instancji w przedziale od $n = 100$ do $n = 500$ wierzchołków zaprezentowano na Rysunku 6.

Algorytm 2-aproksymacyjny charakteryzuje się bardzo dobrą skalowalnością. Na wykresie zbiorczym krzywa reprezentująca ten algorytm przyjmuje postać niemal płaskiej linii, oscylującej tuż nad osią odciętych. Wyniki te potwierdzają, że oparcie aproksymacji wyłącznie na wyznaczaniu MST oraz przeszukiwaniu w głąb (DFS) gwarantuje wysoką wydajność nawet dla relatywnie dużych problemów.

Zupełnie inną dynamikę można zaobserwować w przypadku algorytmu Christofidesa, którego czas wykonania rośnie w sposób gwałtowny i nieliniowy. O ile dla grafów rzędu 100 wierzchołków algorytm ten potrzebuje około 20 ms, o tyle dla instancji największych ($n = 490, n = 500$) czas ten zbliża się do poziomu 350-380 ms. Krzywa odzwierciedla wielomianowy charakter wzrostu złożoności czasowej.

Podobnie jak w przypadku mniejszych instancji, algorytm Christofidesa wykazuje się wysoką wariancją w czasie wykonania — szczególnie zauważalną w przedziale $n \in [350, 500]$, gdzie występują gwałtowne wahania i lokalne ekstrema.

Przedstawione wyniki pokazują, że teoretyczna poprawa współczynnika aproksymacji w algorytmie Christofidesa wiąże się ze znaczącym i szybko rosnącym narzutem obliczeniowym.



Rysunek 6: Porównanie czasu wykonania algorytmów Christofidesa i 2-aproksymacyjnego w funkcji liczby wierzchołków dla dużych instancji losowych ($n \in [100, 500]$).

6.2 Jakość aproksymacji

Do oceny jakości rozwiązań zwracanych przez zaimplementowane algorytmy wykorzystano referencyjny zbiór danych TSPLIB ([examples/tsp-lib/](http://examples.tsp-lib/)), dla którego znane są globalne optima. Pozwoliło to na precyzyjne wyznaczenie **współczynnika aproksymacji**, definiowanego jako stosunek długości trasy wygenerowanej przez algorytm do długości trasy optymalnej. Wyniki eksperymentu, w funkcji rosnącej liczby wierzchołków n , zaprezentowano na Rysunku 7.

Dla ułatwienia analizy, na wykresie naniesiono poziome, przerywane linie w kolorze czerwonym, które reprezentują teoretyczne ograniczenia (górne granice błędu) dla obu analizowanych metod:

- współczynnik 1,5 dla algorytmu Christofidesa oraz
- 2,0 dla algorytmu 2-aproksymacyjnego.

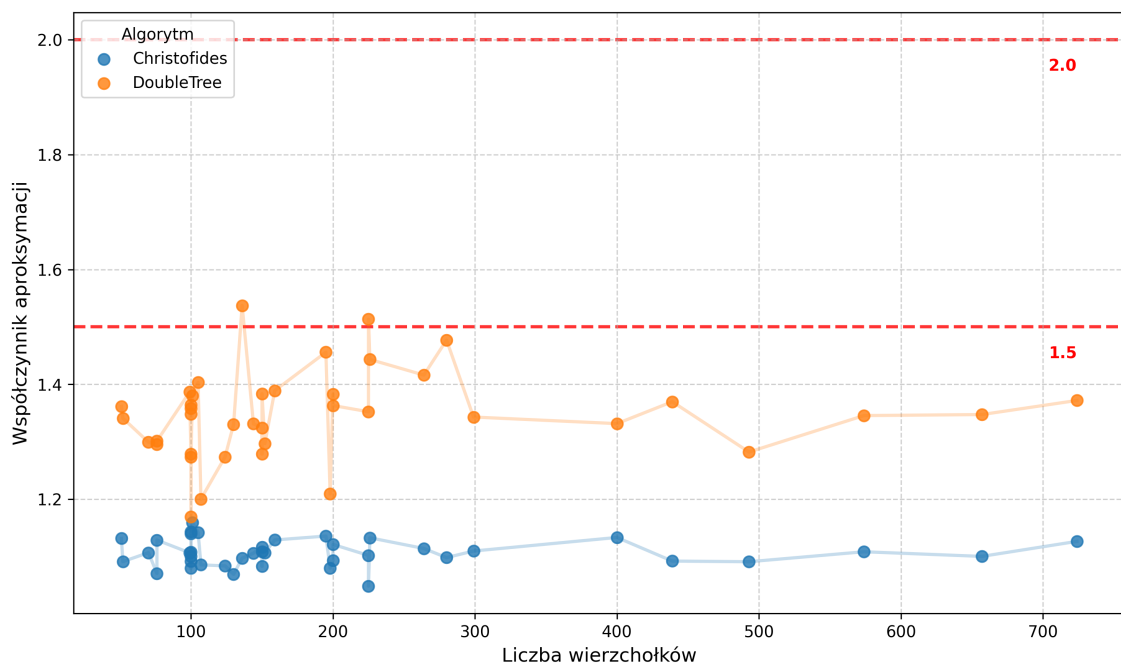
Analiza wykresu pozwala na sformułowanie kilku istotnych wniosków:

1. **Wysoka skuteczność:** Jak wynika z wykresu, algorytm Christofidesa w praktyce radzi sobie zdecydowanie lepiej, niż wskazywałoby na to jego teoretyczne, pesymistyczne ograniczenie

(1,5). Dla badanych instancji TSPLIB, empiryczny współczynnik aproksymacji oscyluje w wąskim paśmie, przyjmując wartości w przybliżeniu od 1,05 do 1,15. Oznacza to, że generowane trasy są zazwyczaj zaledwie o 5–15% dłuższe od znanych optymalnych rozwiązań. Co więcej, wariancja jakości rozwiązań tego algorytmu jest relatywnie niska i nie wykazuje znaczącej tendencji wzrostowej wraz ze zwiększaniem rozmiaru instancji.

2. **Wyższa wariancja i niższa jakość:** Zgodnie z przewidywaniami teoretycznymi, algorytm 2-aproksymacyjny generuje rozwiązania o zauważalnie niższej jakości. Otrzymane współczynniki charakteryzują się większym rozrzutem i mieszczą się przeważnie w przedziale 1,2 – 1,5, przy czym dla pojedynczych instancji wartości te nieznacznie przekraczają próg 1,5. Choć wszystkie wyniki pozostają z dużym marginesem poniżej teoretycznej granicy 2,0, to wygenerowane ścieżki są średnio o 20–50% gorsze od znanych optymalnych rozwiązań. Algorytm ten wykazuje również większą wrażliwość na topologię konkretnych instancji grafowych, co objawia się widocznymi fluktuacjami.

Algorytm 2-aproksymacyjny oferuje zwiększoną szybkość działania i skalowalność wraz z rozmiarem problemu (co udowodniono na dużych instancjach rzędu 500 wierzchołków), jednak odbywa się to kosztem zauważalnej utraty precyzji. Z kolei algorytm Christofidesa gwarantuje znalezienie trasy o wysokiej jakości, bliskiej optimum globalnemu, lecz wymaga zaakceptowania znaczącego wzrostu czasu obliczeń, co w przypadku bardzo dużych instancji może stanowić czynnik dyskwalifikujący.



Rysunek 7: Empiryczny współczynnik aproksymacji w funkcji liczby wierzchołków dla instancji z biblioteki TSPLIB. Czerwonymi przerywanymi liniami zaznaczono teoretyczne ograniczenia algorytmów (1,5 oraz 2,0).

6.3 Analiza statystyczna błędu względnego

Przeprowadzono szczegółową analizę statystyczną procentowego błędu względnego (odchylenia od optimum globalnego) dla referencyjnego zbioru 40 instancji TSPLIB.

Podstawowe statystyki opisowe dla badanych metod zestawiono w Tabeli 1, natomiast Rysunek 8 prezentuje rozkład błędów w postaci wykresów pudełkowych.

Tabela 1: Statystyki opisowe procentowego błędu względnego dla algorytmów Christofidesa i 2-aproksymacyjnego (próba $N = 40$).

Statystyka	Christofides	2-aproksymacyjny
Średnia [%]	10,67	34,76
Mediana [%]	10,66	34,76
Minimum [%]	4,84	16,90
Maksimum [%]	15,90	53,72
Odchyl. stand. [%]	2,34	7,62

Mediana błędu dla algorytmu Christofidesa wynosi zaledwie 10,66%, podczas gdy dla algorytmu 2-aproksymacyjnego osiąga poziom 34,76%. Algorytm Christofidesa charakteryzuje się bardzo zwartym rozkładem wyników – rozstęp międzykwartyłowy jest wąski, odchylenie standardowe wynosi jedynie 2,34%, a maksymalny zaobserwowany błąd nie przekracza 16%.

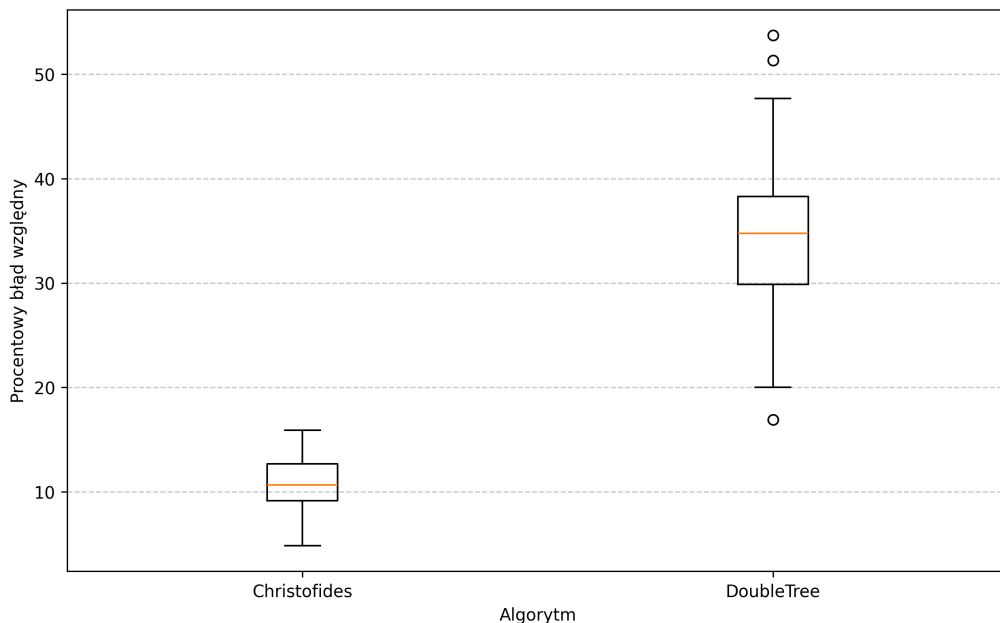
Z kolei algorytm 2-aproksymacyjny wykazuje ponad trzykrotnie większe odchylenie standardowe (7,62%). Na wykresie pudełkowym można zaobserwować obecność wartości odstających (w tym pojedynczych instancji z błędem przekraczającym 50%), co świadczy o znacznej wrażliwości na specyficzną strukturę grafu. Najlepszy wynik dla 2-aproksymacyjnego (błąd 16,90%) wypada gorzej niż najgorszy wynik dla Christofidesa (błąd 15,90%).

6.3.1 Weryfikacja hipotez statystycznych

Aby wykazać istotność statystyczną zaobserwowanych dysproporcji w jakości rozwiązań, przeprowadzono wnioskowanie statystyczne. Ponieważ badane próby są zależne (oba algorytmy testowano na tych samych instancjach problemu), do porównania wykorzystano test Wilcoxon (ang. *Wilcoxon Signed-Rank Test*) [6].

Weryfikacji poddano hipotezę zerową (H_0), zakładającą brak istotnej statystycznie różnicy między medianami błędów względnych obu algorytmów. Przyjęto standardowy poziom istotności $\alpha = 0,05$. Zastosowanie testu dla 40 par wyników pozwoliło na wyznaczenie wartości statystyki testowej $W = 0,0$ oraz odpowiadającej jej wartości prawdopodobieństwa testowego (ang. *p-value*) wynoszącej $p \approx 1,82 \cdot 10^{-12}$.

Uzyskane *p-value* jest znacząco mniejsze od założonego poziomu istotności ($p < \alpha$), zatem **hipoteza zerowa zostaje odrzucona**. Różnica w jakości rozwiązań generowanych przez oba algorytmy jest wysoce istotna statystycznie, z jednoznacznym wskazaniem na algorytm Christofidesa w kontekście precyzji aproksymacji.



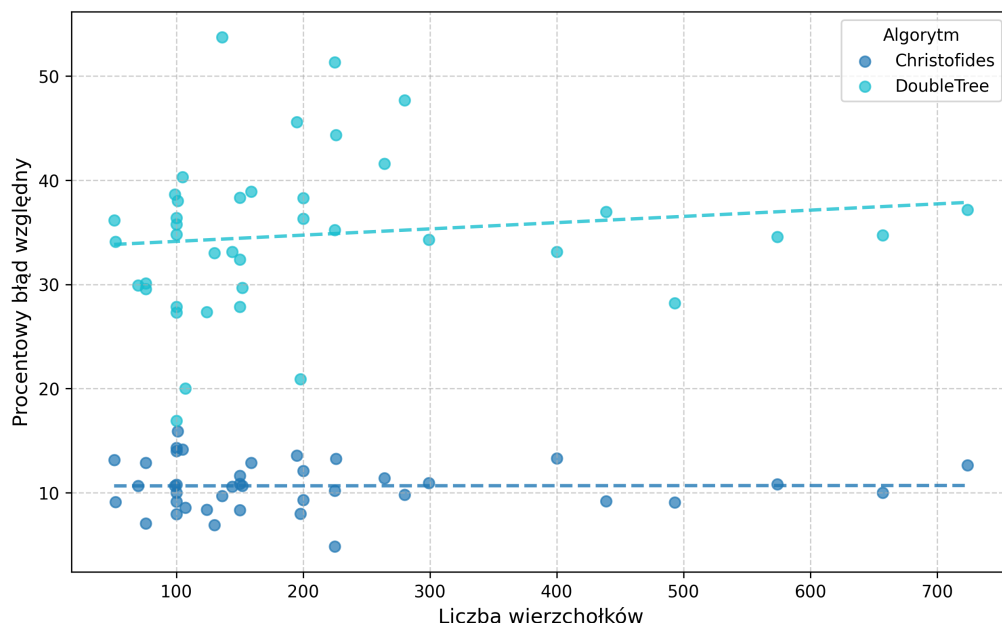
Rysunek 8: Rozkład procentowego błędu względnego dla badanych algorytmów (wykres pudełkowy). Centralna linia pudełka oznacza medianę, krawędzie wyznaczają kwartyły (Q1 i Q3), natomiast punkty reprezentują wartości odstające.

6.4 Zależność błędu aproksymacji od rozmiaru problemu

Z punktu widzenia praktycznych zastosowań algorytmów aproksymacyjnych istotne jest zweryfikowanie, czy jakość zwracanych rozwiązań ulega pogorszeniu wraz ze wzrostem rozmiaru instancji problemu. W tym celu dla instancji z biblioteki TSPLIB wyznaczono linie trendu oraz obliczono współczynniki korelacji między liczbą wierzchołków a procentowym błędem względnym. Wizualizację tej zależności przedstawiono na Rysunku 9.

Analiza metryk statystycznych, w tym współczynnika korelacji liniowej Pearsona (r) [3] oraz korelacji rang Spearmana (ρ) [5], ujawnia odmienną specyfikę badanych metod:

- **Algorytm Christofidesa:** Wykazuje niemal całkowity brak korelacji pomiędzy rozmiarem grafu a wielkością popełnianego błędu ($r = 0,0040$, $\rho = 0,0123$). Linia trendu dla tego algorytmu jest praktycznie pozioma i stabilnie oscyluje wokół wartości 10–11%. Algorytm potrafi utrzymać wysoką precyzję niezależnie od tego, czy przetwarza graf o kilkudziesięciu, czy kilkuset wierzchołkach.
- **Algorytm 2-aproksymacyjny:** Wykazuje słabą korelację dodatnią ($r = 0,1289$, $\rho = 0,2804$). Choć linia trendu zauważalnie wznosi się wraz ze wzrostem liczby wierzchołków, zależność ta nie jest na tyle silna, by dyskwalifikować algorytm dla dużych instancji. Potwierdza to jednak wcześniejsze obserwacje o wyższej wariancji i ogólnej mniejszej stabilności optymalizacyjnej algorytmu opartego wyłącznie na podwójnym drzewie rozpinającym.



Rysunek 9: Wykres rozrzutu ilustrujący zależność procentowego błędu względnego od liczby wierzchołków wraz z wyznaczonymi liniami trendu.

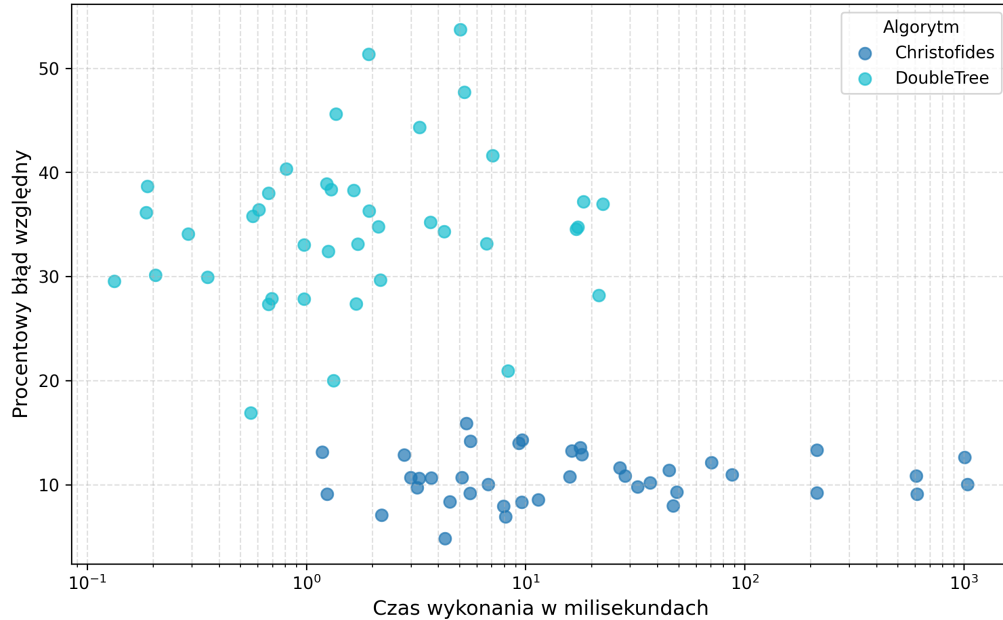
6.5 Czas obliczeń a precyzja

Ostatecznym podsumowaniem przeprowadzonych eksperymentów jest bezpośrednie zestawienie dwóch rywalizujących ze sobą wskaźników wydajnościowych: czasu wykonania oraz wielkości procentowego błędu względnego. Rysunek 10 prezentuje to zestawienie w postaci wykresu rozrzutu. Ze względu na różnice w narzucie obliczeniowym obu algorytmów (różnice rzędu kilku wielkości), oś odciętych (czas w milisekundach) została przedstawiona w skali logarytmicznej.

W przestrzeni wyników formują się dwie, wyraźnie odseparowane od siebie chmury punktów:

1. **Szybkość kosztem jakości:** Punkty reprezentujące algorytm 2-aproksymacyjny są wyraźnie przesunięte do lewej krawędzi wykresu (bardzo niskie czasy wykonania, od ułamków do kilkunastu milisekund), wykazując jednocześnie duży rozrzut wzdłuż osi rzędnych (błędy w przedziale 20–55%). Jest to algorytm dedykowany dla systemów, w których szybkość podjęcia decyzji jest priorytetem, a gorsza jakość wyznaczonej trasy jest kosztem w pełni akceptowalnym.

2. **Jakość kosztem czasu:** Reprezentacja wyników dla algorytmu Christofidesa tworzy wąskie, spłaszczone pasmo w dolnej sekcji wykresu (błędy utrzymane konsekwentnie poniżej 16%), które jednak rozciąga się w prawo na osi logarytmicznej (osiągając czasy rzędu 10^3 ms). Implementacja ta jest zatem uzasadniona w zastosowaniach, w których koszt samego procesu obliczeniowego jest pomijalny w stosunku do realnych oszczędności wynikających z odnalezienia trasy bliskiej optimum.



Rysunek 10: Zestawienie kompromisu pomiędzy czasem wykonania (skala logarytmiczna) a procentowym błędem względnym dla referencyjnych instancji z biblioteki TSPLIB.

7 Podsumowanie

W ramach projektu zaimplementowaliśmy i porównaliśmy dwa algorytmy aproksymacyjne rozwiązujące metryczny problem komiwożacza (TSP): algorytm 2-aproksymacyjny (wykorzystujący podwójne drzewo rozpinające) oraz algorytm Christofidesa. Ponadto napisaliśmy własny generator instancji testowych oraz stworzyliśmy narzędzia do automatyzacji badań i wizualizacji wyników.

Aby ocenić skuteczność obu metod, przeprowadziliśmy serię eksperymentów na trzech zestawach danych: małych i dużych grafach wygenerowanych losowo (od 10 do 500 wierzchołków) oraz na referencyjnych instancjach z biblioteki TSPLIB. Zbadaliśmy przede wszystkim dwa aspekty: czas działania algorytmów oraz jakość wyznaczanych tras (czyli to, jak bardzo znaleziona ścieżka różni się od tej optymalnej).

Nasze badania pokazały wyraźny kompromis między szybkością a precyzją obliczeń. Najważniejsze wnioski płynące z eksperymentów są następujące:

- **Algorytm 2-aproksymacyjny jest bardzo szybki i świetnie się skaluje.** Rozwiązywanie problemów dla grafów o wielkości 500 wierzchołków zajmuje mu zaledwie kilka milisekund. Kosztem tej szybkości jest jednak niższa dokładność – wyznaczone trasy są średnio o 35% dłuższe od optymalnych, a same wyniki bywają niestabilne i mocno zależą od struktury grafu.
- **Algorytm Christofidesa daje dużo lepsze i stabilniejsze wyniki.** Znalezione przez niego rozwiązania są średnio zaledwie o 10–11% dłuższe od optimum. Osiąga to jednak kosztem znacznie dłuższego czasu działania, który gwałtownie rośnie wraz z wielkością problemu (głównie ze względu na konieczność wyznaczania skojarzenia doskonałego).

Testy statystyczne potwierdziły, że różnica w jakości między tymi algorytmami nie jest przypadkowa. Wybór konkretnej metody zależy więc od potrzeb. Jeśli zależy nam na natychmiastowej odpowiedzi systemu, lepszym rozwiązaniem będzie algorytm 2-aproksymacyjny. Z kolei w sytuacjach, gdzie zależy nam na wyznaczeniu trasy jak najbliższej optimum i możemy poczekać na wynik, warto postawić na algorytm Christofidesa.

8 Podział pracy

Osoba	Zadania
Marcin Cieszyński	• Analiza jakości aproksymacji i złożoności obliczeniowej dla algorytmu 2-aproksymacyjnego. • Implementacja algorytmu 2-aproksymacyjnego. • Uruchomienie programu na instancjach testowych i zebranie wyników do plików CSV. • Analiza wyników eksperymentów.
Radosław Głasek	• Analiza jakości aproksymacji i złożoności obliczeniowej dla algorytmu Christofidesa. • Analiza teoretyczna algorytmu Hierholzera oraz algorytmu Edmonds'a w wariancie pierwotno-dualnym. • Implementacja algorytmu Christofidesa. • Analiza wyników eksperymentów.
Adam Grącikowski	• Specyfikacja oraz implementacja wejścia i wyjścia. • Implementacja generatora instancji testowych. • Wygenerowanie instancji losowych oraz przygotowanie instancji testowych z biblioteki TSPLIB do przeprowadzenia eksperymentów. • Instrukcja użytkownika wraz z przykładami uruchomienia. • Przygotowanie skryptów generujących wykresy dla wyników eksperymentów. • Analiza wyników eksperymentów.

Bibliografia

- [1] Marcin Cieszyński, Radosław Głasek i Adam Grącikowski. *Porównanie dwóch algorytmów aproksymacyjnych znajdujących rozwiązanie problemu Komiwożazera. Dokumentacja wstępna. Dokumentacja projektowa. Projekt z przedmiotu: Algorytmy zaawansowane. Wydział Matematyki i Nauk Informacyjnych, Politechnika Warszawska, maj 2026.*
- [2] mastqe. *TSPLIB: Zbiór instancji problemu komiwożazera*. <https://github.com/mastqe/tsplib>. [Dostęp online: 17-05-2026].
- [3] Karl Pearson. „Note on regression and inheritance in the case of two parents”. W: *Proceedings of the Royal Society of London* 58 (1895), s. 240–242.
- [4] Gerhard Reinelt. „TSPLIB—A Traveling Salesman Problem Library”. W: *ORSA Journal on Computing* 3.4 (1991), s. 376–384.
- [5] Charles Spearman. „The Proof and Measurement of Association between Two Things”. W: *The American Journal of Psychology* 15.1 (1904), s. 72–101.
- [6] Frank Wilcoxon. „Individual Comparisons by Ranking Methods”. W: *Biometrics Bulletin* 1.6 (1945), s. 80–83.